

INTRODUCTION TO INFO. & COMM. TECHNOLOGY

22-11-2023

Syntax

- Coding requires using the programming language software to enter the program statements into the computer's memory (and to save them to a storage medium)
- The ways that specific words and symbols are used by each language is called its **syntax**

Syntax

- The actual code→the exact words and symbols used to write the code
- Syntax depends on what language you are using
- the rules that govern the structure of the language

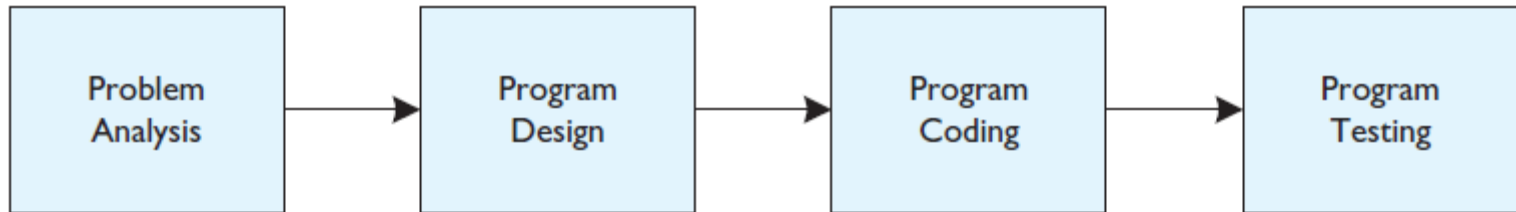
Testing the Program

- To ensure that the program is free of errors and that it solves the given problem
- When you run the program → have the computer execute its statements—using various sets of input data to determine if the program is working properly
- Testing (or checking) should take place throughout the development process and not just at the end

Testing

- In the design phase, try to run through the code using simple input data to see if you get the expected results
- This is referred to as **desk-checking** or walking through a program

The program development process



Types of Errors

- If a test run turns up problems with the program, we must **debug** it
- Locate and eliminate the errors
- The two fundamental types of errors that can arise in coding a program:
 - Syntax errors
 - Logic errors

Syntax Errors

- It is the violation of the programming language's rules for creating valid statements
- **Reasons:**
- Misspelling a keyword
- Omitting a required punctuation mark

Syntax Errors

- These are detected by the language software, either when the invalid statement is typed or when the program is translated (compiled) by the computer into machine language
- On a syntax error → software issues a message and highlights the statement
- Sometimes the software finds an error in one place that actually was caused by another type of error somewhere else in the code
- → analyze the error message, find the place where the error originates, and correct it

Logic Errors

- This type of error results from failing to use the proper combination of statements to accomplish a certain task
- Reasons:
 - Faulty analysis
 - Faulty design
 - Failure to code the program properly

Logic Errors

- Examples of Logic errors:
- Use of an **incorrect formula** to produce a desired result
- Use of an **incorrect sequence** of statements
- Failure to predict some **input data** that may lead to an illegal operation (such as division by zero) → called a **runtime error**

Logic Error

- Logic errors often cause the program to fail to proceed beyond a certain point (i.e., to crash or hang or freeze) or to give incorrect results
- Logic errors are not detected by the programming language software
- These can be found only by running the program with a sufficient variety of test data
- Extensive testing is needed to ensure that a program's logic is sound

Condition

- Condition is an expression that is either false (represented by 0) or true (represented by 1)
 - e.g., `Marks > 50` is a condition
- Conditions may contain relational or equality operators, and have the following forms:

variable relational-operator variable (or constant)

variable equality-operator variable (or constant)

Relational Operators

Table 4.1 Relational operators

Operator	Description
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to (is same as)
!=	is not equal to

Relational Operators

- In programming, there is a distinction between the equals sign (=) and a double equals sign (==)
- The equals sign is an **assignment operator**
- The double equals sign is called a **comparison operator**
- We use the symbol == (a double equals sign) when we compare the value of a variable with another variable, value, or expression

Relational Operators

- Relational operators allow you to compare numeric and char values and determine whether one is greater than, less than, equal to, or not equal to another

Relational Operators

- All the relational operators are binary operators with left-to right associativity
- Associativity is the order in which an operator works with its operands
- Binary operators $\rightarrow$ they use two operands

$a > b$

$x < 10$

The above expressions are called *relational expressions*

Value of a Relationship

- All expressions have a value
- Relational expressions are *Boolean expressions* → their value can only be *true* or *false*
- If *x* is greater than *y* , the expression $x > y$ will be true and the expression $x < y$ will be false.

Value of a Relationship

- The `==` operator determines whether the operand on its left is equal to the operand on its right
- If both operands have the same value, the expression is true
- Let's assume that `a is 4`, the following expression will return TRUE:

`a == 4`

Value of a Relationship

- Two relational operators test for a pair of relationships
- The $\geq$ operator determines whether the operand on its left is greater than *or* equal to the operand on the right
- If a is 4, b is 6, and c is 4, both of the following expressions are TRUE:

$b \geq a$

$a \geq c$

$b \leq a$??

Value of a Relationship

- Two relational operators test for a pair of relationships
- The $\geq$ operator determines whether the operand on its left is greater than *or* equal to the operand on the right
- If a is 4, b is 6, and c is 4, both of the following expressions are TRUE:

$b \geq a$

$a \geq c$

the following expression is FALSE:

$b \leq a$

Value of a Relationship

- `!=` operator is the not-equal operator
- It determines whether the operand on its left is different than (i.e., *not* equal to) the operand on its right
- If a is 4, b is 6, and c is 4, both of the following expressions are TRUE:

`a != b`

`b != c`

`a != c` **????**

Value of a Relationship

- `!=` operator is the not-equal operator
- It determines whether the operand on its left is different than (i.e., *not* equal to) the operand on its right
- If a is 4, b is 6, and c is 4, both of the following expressions are TRUE:

`a != b`

`b != c`

`a != c` **False**

Value of a Relationship

- How does a computer store *true* in memory?
- How does it store *false*?
- These values are converted to numbers
- In C++ zero is considered false and any nonzero value is considered true
- So the C++ keyword *false* is stored as 0, and the keyword *true* is stored as 1
- When a relational expression is false, it evaluates to 0
- When a relational expression is true, it does not always evaluate to 1
- It can actually evaluate to any nonzero value

Let's see through the code - Ex-1

```
#include <iostream>
using namespace std;

int main()
{
    // This program displays the values C++ uses to represent true and false.

    bool trueValue, falseValue;
    int x,y;
    x = 5;
    y = 10;
    trueValue = (x < y);
    falseValue = (y == x);
    cout << "True is " << trueValue << endl;
    cout << "False is " << falseValue << endl;
    return 0;
}
```

The run window

```
True is 1
False is 0

-----

Process exited after 0.07523 seconds
Press any key to continue . . .
```

- Parentheses are not actually required
- Relational operators have a higher precedence than the assignment operator
- Arithmetic operators have a higher precedence than relational operators

Ex-2

- The statement
`result = x < y - 8;`

- is equivalent to the statement

`result = x < (y - 8);`

- In both cases, $y - 8$ is evaluated first
- Then this value is compared to x
- So it is always a good idea to use parentheses

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

`x == 5`

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

`x == 5`

TRUE

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

TRUE

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

TRUE

$(2 + x) \neq y$

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

TRUE

$(2 + x) \neq y$

TRUE

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

TRUE

$(2 + x) \neq y$

TRUE

$z \neq 4$

Ex-3

- Assuming x is 5, y is 6, and z is 8, indicate whether each of the following relational expressions is true or false:

$x == 5$

TRUE

$7 \leq (x + 2)$

TRUE

$z > 4$

TRUE

$(2 + x) \neq y$

TRUE

$z == 4$

FALSE

Logical Operators

- These are used to create compound conditions from given simple conditions
- The most important logical operators
- `||` → logical OR
- `&&` → logical AND
- `!` → logical NOT
- Logical expression uses one or more logical operators

Logical Operators

- `!`
Reverses the truth value of the condition, `True` → `False`,
`False` → `True`
- `&&`
- It takes two expressions as operands and creates an expression that is `true` only when `both` subexpressions are `true`
- `||`
- It takes two expressions as operands and creates an expression that is `true` when `either` of the subexpressions is `true`

Truth Tables for the OR, AND, and NOT Operators

Table 4.2 Truth Tables for the OR, AND, and NOT operators

X	Y	X OR Y	X AND Y	NOT X
true	true	true	true	false
true	false	true	false	false
false	true	true	false	true
false	false	false	false	true

Don't forget to study table 4.3 for complete details on hierarchy of operators

Increment and Decrement

- Changing a variable by adding or subtracting 1
- A special shorthand for it

`counter++;`

`counter--;`

- The ++ increment operator gave the C++ programming language its name
- C++ is the incremental improvement of the C language

Let's look at an example

```
#include <iostream>
using namespace std;
int main()
{
    int b,x;
    b=10;
    x=2;
    b = x++;
    cout<<"Value in B = "<<b<<endl;
    cout<<"Now value in X = "<<x<<endl;

    b = ++x;
    cout<<"This time value in B is different"<<endl;
    cout<<"Value in B = "<<b;

    return 0;
}
```

Run Window

 E:\My Course Books\ICT\C++File-F2023\Lec13-Ex1.exe

```
Value in B = 2
```

```
Now value in X = 3
```

```
This time value in B is different
```

```
Value in B = 4
```

```
-----
```

```
Process exited after 9.964 seconds with return value 0
```

```
Press any key to continue . . .
```

Data Types

- Three types of data types:
 - Simple Data Type
 - Structured Data Type
 - Pointers

Simple Data types

- Fundamental data type in C++
- Integral → to deal with integers, numbers without decimal part
- Floating point → to deal with decimal numbers
- Enumeration → which is a user defined data type (to allow programmers to create their simple data types)→TBDL

Integers

- Positive integers do not need a + sign in front of them
- No commas to be used

Bool Data type

- It has two values, true and false. Also called logical values
- Will print 0 in output as it is false
- We can use bool type variables or values true and false in mathematical expressions also
- $x = 7$ will be displayed as output

```
bool my_flag;  
my_flag = false;  
cout <<"Flag = " <<my_flag<<endl;
```

```
int x = false + true + 6;  
cout<<"x = " <<x<<endl;
```

char Data type

- To represent single character → letters, numbers and special symbols
- Enclose each character within single quotation marks
- 'A', 'a', '0', '*', '+', '\$', '&', ' '
- Note that a blank space is a character written as ' '
- the value 'abc' is not of the type char

Selection Construct

- Selection Structures
- Decision Structures
- Types
 - Single Alternative (If-Then) structure
 - Dual Alternative (If-then-Else) structure
 - Multiple Alternative (Select-Case) structure

Ex-4

```
#include <iostream>
using namespace std;
const int SECRET = 5;
int main()
{
    int x, y, w, z;
    cout<<" Enter a value for variable z:  ";
    cin>>z;

    if (z > 10)
    {
        x = 12;
        y = 5;
        w = x + y + SECRET;
    }
    else
    {
        x = 12;
        y = 4;
        w = x + y + SECRET;
    }
    cout << "w = " << w << endl;
    return 0;
}
```

Ex-4

```
Enter a value for variable z: 10
w = 21

-----
Process exited after 2.289 seconds with return value 0
Press any key to continue . . .
```

```
Enter a value for variable z: 13
w = 22

-----
Process exited after 10.58 seconds with return value 0
Press any key to continue . . .
```

```
Enter a value for variable z: 2
w = 21

-----
Process exited after 2.092 seconds with return value 0
Press any key to continue . . .
```

Ex-5

- Write a C++ code that accepts years of experience for an employee. If the experience is greater than 5 years, bonus is 10% of the salary. If the experience is less than 5 years, then bonus is 5% of the salary. User enters salary as well.

Ex-6

- Ask the user to enter temperature of water in degree Celsius ($^{\circ}\text{C}$). The program displays “FROZEN” if it is less than zero. It displays “BOILING” if it is equal to 100. Otherwise it displays “NO PROBLEM”

Ref Chapters

- Book 3
- Chp 1,2